Self-describing GOAL traces for LogGOPSim

DPHPC 2026 progress presentation

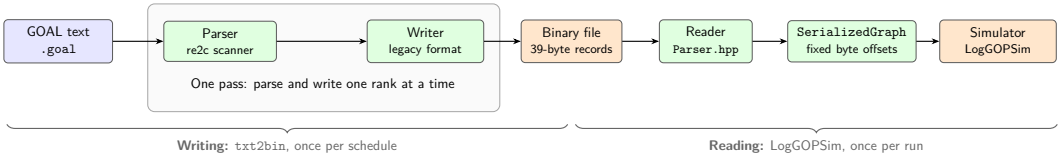
Joshua Durrant, Alexander Falter, Samuel Garcin, Mykhaylo Gershman

Supervised by Timo Schneider

17.11.26

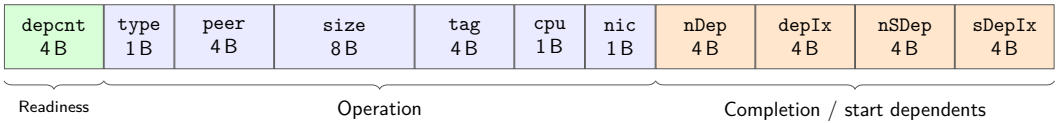
LogGOPSim today

LogGOPSim predicts how long an MPI program runs. Input: per rank, a graph of sends, receives and computations. Output: the predicted runtime.



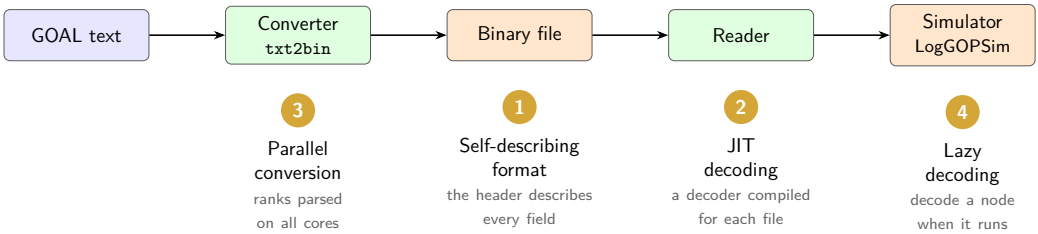
- `txt2bin` converts the text schedule once; each simulation then reads the binary file.
- The reader is built into LogGOPSim and knows the record layout by heart.

The problem: a fixed 39-byte record



- **Wasteful:** every field at full width, whatever values occur. Most traces need 2–4 bytes per node.
- **Closed:** no room for extra information. ATLAHS hides the GPU in `nic` and packs five numbers into the tag; a wrong guess gives wrong results silently.
- **Hard-wired:** each simulator needs its own copy of the byte offsets.

Our design in four steps

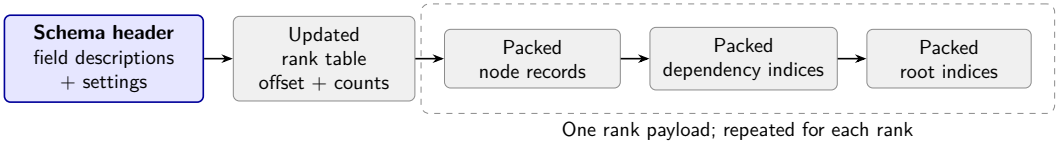


Step 1

Self-describing format

The file says what is in it

A header that describes every field



- Each field gets only the bits its values need. The header stores a base, records store the offset: a constant field takes **0 bits**.
- **Custom fields** (declared in TOML) are treated like standard ones; each simulator binds only the fields it needs.
- Checked at load time: a missing field or a value that doesn't fit is an error, not a wrong result.
- Legacy files stay readable through the same reader.

Custom fields in practice: ATLAHS

```
before: l2: send 1024b to 1 tag 3050012001 cpu 0 nic 2
after:  l2: send 1024b to 1 tag 0 cpu 0 nic 0 stream_id 0 gpu_id 2 peer_gpu_id 2
            chunk_id 3 channel_id 5 sequence_id 12 message_kind 0 communicator_id 1
```

- Eight explicit fields instead of guesses.
- Fixed two htsim bugs: messages sent to the wrong GPU, and different messages sharing a tag.

Recorded Llama run, 5.7M operations:

54% smaller

with all eight extra fields

Step 2

JIT decoding

A flexible format must not be slower to read

Compile a decoder for each file

```
int decode(const u8* r, u8* const* t) {
  u64 w, c;
  // peer: 16 bits at bit 3
  memcpy(&w, r + 0x0, 8);
  c = (w >> 0x3) & 0xffff;
  store32(t[0] + 0x3c, c);
  // size: 0 bits, constant 1024
  store64(t[0] + 0x40, 0x400);
  // ... further fields
  return 1;
}
```

Generated C for bcast_64k, abridged.

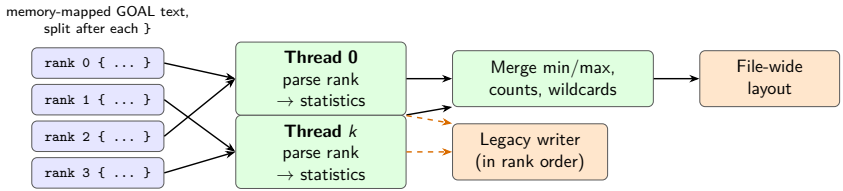
- Bit positions are known only when the file is opened. A generic decoder took up to **47%** of the simulation time.
- Once per file: build a decode plan from the header and the simulator's fields, then generate code in which every position and mask is a constant.
- `jit`: compiled with `cc`, cached on disk. `orc`: compiled in memory with LLVM.
- Fallback: the plan interpreter.

Step 3

Parallel conversion

Parse ranks on all cores

The converter's first pass, in parallel



- The new format needs every field's range before writing, so `txt2bin` parses the text twice.
- Ranks are independent: the mapped text is cut after each rank and parsed on OpenMP threads.
- Per-thread statistics merge in any order: same bytes, same errors as the sequential pass. `txt2bin --threads N`.

Step 4

Lazy decoding

Decode a node only when the simulator runs it

Decode once, never write the file

	Before	`--lazy-decoding`
Ready node	decoded copy, 80 bytes	index, 4 bytes
Decoded	when it becomes ready	once, when it runs
Dependency counts	written into the mapped file; every written page is copied	array in memory, 4 bytes per node

- Same simulation: the full event log is identical.
- Works with both formats and every decoder.

Files are 53–93% smaller

Trace	Bytes per node	File size	Smaller
bcast_64k	39 → 4	6.9 → 1.9 MiB	73%
dissem_16k	39 → 4	19.2 → 2.4 MiB	88%
alltoall_256	39 → 2	5.4 MiB → 404 KiB	93%
wide_64k (random 64-bit values)	39 → 15	6.9 → 3.2 MiB	53%
ATLAHS Llama run (+8 fields)	39 → 17	240.6 → 111.4 MiB	54%

Narrow and constant fields shrink most; even random 64-bit values save half.

Simulations are faster than the original

Trace	Original	Ours, legacy file	Ours, new + `jit`	`jit` vs. original
bcast_64k	48.4	43.9	43.7	−10%
dissem_16k	146.4	122.7	110.8	−24%
ring_256	39.8	32.8	32.9	−17%
alltoall_256	1950	2004	1996	+2%
wide_64k	crashes	78.4	79.5	

- Complete LogGOPSim runs in ms, identical results. alltoall is dominated by message matching.
- Decoding alone: `jit` is about 2× faster than the interpreter and within 6% of code compiled ahead of time.

Apple M3 Pro, median of 21 interleaved runs. Original: last upstream commit (76451de).

Parallel conversion and lazy decoding

Converter pass 1, seconds

Trace	Seq.	8 threads	
dissem_65536	2.79	0.60	**4.6**×
ring_2048	22.1	8.6	**2.6**×
alltoall_4096	38.1	10.7	**3.6**×

Peak memory of a simulation, MB

Trace	Before	Lazy	
dissem_524288	2220	1540	−31%
ring_2048	1647	1103	−33%
resnet_8192	216	153	−29%

- Two threads nearly halve pass 1; beyond four it flattens. Pass 2 is still sequential.
- Lazy decoding is also 2–11% faster.

WSL2 with 8 vCPUs, median of 3 runs. Lazy: new format, `plan` decoder; peak RSS.

Outlook

Next optimisations

- Parallelise pass 2 and the legacy writer.
- Dictionary encoding for few, far-apart values.
- Choose the decoder automatically from trace size and startup cost.
- Full benchmark study on Euler.

Conversion on the GPU (CUDA)

- Ranks and nodes are independent: one GPU thread per node instead of one CPU thread per rank.
- Field statistics are min/max reductions; packing records is independent per node.
- GPU memory bandwidth for streaming very large traces.

Who did what

Name	Work
Joshua Durrant	Full benchmarking study (upcoming)
Alexander Falter	Parallel conversion (OpenMP first pass, `--threads`, tests, benchmark); faster legacy writer; lazy decoding
Samuel Garcin	Decode plan, C JIT with cache, LLVM ORC JIT, decode benchmark, differential tests
Mykhaylo Gershman	Self-describing format and converter, custom fields, validating reader and adapters, ATLAHS integration, runner architecture, test and benchmark suites, report

Questions?